```
env = magent.GridWorld("battle", map_size=map_size)    2
env.set_render_dir("MAgent/build/render")              3

team1, team2 = env.get_handles()                       4
```

- ***1*** Imports the cityblock distance function from scipy to compute distances between agents on the grid
- ***2*** Sets up the environment in "battle" mode with a 30 x 30 grid
- ***3*** Sets up our ability to view the game after training
- ***4*** Initializes the two team objects

MAgent is highly customizable, but we will be using the built-in configuration called "battle" to set up a two-team battle scenario. MAgent has an API similar to OpenAI Gym but there are some important differences. First, we have to set up "handles" for each of the two teams. These are objects, `team1` and `team2`, that have methods and attributes relevant to each team. We generally pass these handles to a method of the environment object, `env`. For example, to get a list of the coordinates of each agent on team 1, we use `env.get_pos(team1)`.

We're going to use the same technique to solve this environment as we did for the 2D Ising model, but with two DQNs. We'll use a softmax policy and experience replay buffer. Things will get a bit complicated because the number of agents changes over training, since agents can die and be removed from the grid.

With the Ising model, the state of the environment was the joint actions; there was no additional state information. In MAgent we additionally have the positions and health points of agents as state information. The Q function will be $Q_j(s_t,a_{-j})$ where a_{-j} is the mean field for the agents within agent j's *field of view* (FOV) or neighborhood. By default, each agent has a FOV of the 13 × 13 grid around itself. Thus, each agent will have a state of this binary 13 × 13 FOV grid that shows a `1` where there are other agents. However, MAgent separates the FOV matrix by teams, so each agent has two 13 × 13 FOV grids: one for its own team and one for the other team. We will need to combine these into a single state vector by flattening and concatenating them together. MAgent also provides the health points of the agents in the FOV, but for simplicity we will not use these.